

聚类算法对比实验

实验原理

K-means

K-means算法是一种聚类算法。算法开始时，在数据集中随机选择K个数据点作为聚类中心。随后，计算剩下的点到聚类中心的距离，并将这些点分配到最近的聚类中心中。分配完成后，计算各个簇中数据点的均值，作为新聚类中心的坐标。重复这个过程，直到每次迭代前后聚类中心位置的变化值小于设定值。

DBSCAN

DBSCAN算法也是一种聚类算法。算法开始时，在数据集中任意选择一个数据点作为种子，然后开始寻找这个数据点的“邻居”，“邻居”的寻找范围由使用者手动指定。这个点和它的“邻居”，就是一个聚类簇。随后，再寻找一个不在任何簇中的点，寻找它的邻居，形成它的簇，直到所有点都有它的分类（在某个簇中，在簇的边界中，或不能和其他任何一个点形成簇）

实验过程

数据集

本次实验使用的鸢尾花数据集可以直接在 `sklearn` 中导入。

```
iris_dataset = load_iris()
X = iris_dataset.data
y = iris_dataset.target
```

K-means实现逻辑

```
def kmeans(X, y, k):
    n_samples, n_features = X.shape
    centroids = X[np.random.choice(n_samples, k, replace=False)]

    for i in range(300):
        # 每次迭代计算距离，根据距离确定标签
        distances = np.sum((X[:, np.newaxis] - centroids) ** 2, axis=2)
        labels = np.argmin(distances, axis=1)
        centroids = []
        for i in range(k):
            if np.any(labels == i):
                centroids.append(X[labels == i].mean(axis=0))
            else:
                centroids.append(X[np.random.choice(n_samples)])
    return centroids, labels
```

K-means算法的实现逻辑是：

首先，初始化样本数和特征数。随机在数据集中选择K个不重复的点，作为质心数组。

随后，开始迭代。

每次迭代时，先计算每个数据点到各个质心的距离。具体的计算方法是，对每个数据点和每个质心，求它们之间各个维度的差值的平方之和，这就得到了每个数据点到每个质心的距离。

根据计算结果，求得每个数据点对应的标签。

清空质心数组，重新计算质心数组。对每个类别，计算这种类别的数据点的每个特征的均值，作为新的质心。

由于在本实验中，数据集较小，是否添加退出条件（两次质心位置变化小于设定值就提前退出）对效率影响不大，故没有设置。

DBSCAN实现逻辑

```
def Near(X, eps, i):  
    return np.where(np.linalg.norm(X - X[i], axis=1) <= eps)[0]
```

DBSCAN算法中很重要的一部分就是寻找每个点的邻居。以上是寻找邻居的函数，会返回到输入点距离小于设定值的点的numpy数组。

```
def cluster(X, eps, sample, i, near, id, labels):  
    labels[i] = id  
    queue = deque(near)  
    while queue:  
        point = queue.popleft()  
        if labels[point] != -1:  
            continue  
        labels[point] = id  
        near_new = Near(X, eps, point)  
        queue.extend(set(near_new) - set(queue))
```

聚类逻辑也同样重要。对每个未分类的点，给它一个新的类，建成一个它的邻居队列。对于每个邻居，如果它已经有分类，就跳过它；如果没有分类，就将邻居添加到它的类中。在迭代中，寻找邻居的邻居，并将新邻居添加到邻居队列中。

```

def DBSCAN(X, y, eps, sample):
    sample_num = X.shape[0]
    labels = -1 * np.ones(sample_num)
    id = 0

    for i in range(sample_num):
        if labels[i] != -1:
            continue
        near = Near(X, eps, i)
        if len(near) < sample:
            labels[i] = -1
        else:
            cluster(X, eps, sample, i, near, id, labels)
            id += 1

    return labels

```

DBSCAN算法中，首先初始化标签为-1，随后对每个未分类的样本，计算它的邻居。如果它的邻居数不足以让它成为核心点，就跳过它；否则，以它为核心进行聚类。

实验结果

评估方法

```

1 usage
def kmeans_evaluate(X, y, k):
    centroids, labels = kmeans(X, y, k)
    accuracy = accuracy_score(y, labels)
    ari = adjusted_rand_score(y, labels)
    silhouette = silhouette_score(X, labels)
    ch_score = calinski_harabasz_score(X, labels)
    return accuracy, silhouette, ch_score, ari

1 usage
def DBSCAN_evaluate(X, y, eps, sample):
    labels = DBSCAN(X, y, eps, sample)
    labels_valid = labels[labels != -1]
    X_valid = X[labels != -1]
    accuracy = accuracy_score(y, labels)
    ari = adjusted_rand_score(y, labels)
    silhouette = silhouette_score(X_valid, labels_valid)
    ch_score = calinski_harabasz_score(X_valid, labels_valid)
    return accuracy, silhouette, ch_score, ari

```

在评估DBSCAN算法结果时，需要使用有分类的X和标签计算轮廓系数与Calinski-Harabasz指数。

由于在聚类时，聚类结果的类标签是随机的，即同一批样本，在原始数据中都是属于类1，在聚类后可能都被分进类9，这对准确率计算造成了很大的问题。因此，我采用了ARI作为轮廓系数、CH指数之外的评价标准。

评估结果

```

C:\Python312\python.exe E:\学习\Pycharm\Cluster\main.py
K-means (k=2) - Accuracy: 0.647, Silhouette: 0.681, Calinski-Harabasz: 513.925, ARI:0.5399218294207123
K-means (k=3) - Accuracy: 0.887, Silhouette: 0.551, Calinski-Harabasz: 561.594, ARI:0.7163421126838476
K-means (k=5) - Accuracy: 0.367, Silhouette: 0.373, Calinski-Harabasz: 459.506, ARI:0.49478421932222
DBSCAN (eps=0.3, sample = 3) - Accuracy: 0.273, Silhouette: 0.616, Calinski-Harabasz: 320.279, ARI:0.2818190729052577
DBSCAN (eps=0.3, sample = 5) - Accuracy: 0.387, Silhouette: 0.761, Calinski-Harabasz: 510.531, ARI:0.29923870749450177
DBSCAN (eps=0.3, sample = 7) - Accuracy: 0.347, Silhouette: 0.830, Calinski-Harabasz: 569.076, ARI:0.3196543652284484
DBSCAN (eps=0.5, sample = 3) - Accuracy: 0.620, Silhouette: 0.424, Calinski-Harabasz: 258.174, ARI:0.5226122364041716
DBSCAN (eps=0.5, sample = 5) - Accuracy: 0.620, Silhouette: 0.735, Calinski-Harabasz: 680.486, ARI:0.5206185241703302
DBSCAN (eps=0.5, sample = 7) - Accuracy: 0.620, Silhouette: 0.735, Calinski-Harabasz: 680.486, ARI:0.5206185241703302
DBSCAN (eps=0.7, sample = 3) - Accuracy: 0.667, Silhouette: 0.694, Calinski-Harabasz: 529.625, ARI:0.5621364251426576
DBSCAN (eps=0.7, sample = 5) - Accuracy: 0.667, Silhouette: 0.694, Calinski-Harabasz: 529.625, ARI:0.5621364251426576
DBSCAN (eps=0.7, sample = 7) - Accuracy: 0.667, Silhouette: 0.694, Calinski-Harabasz: 529.625, ARI:0.5621364251426576

Process finished with exit code 0

```

在K-means算法中，K=3时效果最好。

在DBSCAN算法中，eps = 0.3, 最小样本数为5或7时效果最好。